

RAG vs Agent in Production: Cost and p99, by Hand

Why a RAG request has *one* price and an agent run has a *distribution* — and why you must provision for the p99 tail, not the average.

What this document does. It takes the two production shapes from Module 4.0 — a *RAG request* (one retrieval + one LLM call, a fixed pipeline) versus an *agent trajectory* (the model decides when to stop, so the step count N varies) — and computes their dollar cost and latency *by hand*. We fix the token sizes, derive the single-call RAG cost ($\approx \$0.005$), then build a small 20-trajectory sample of agent runs with a heavy right tail, and compute its mean, $p50$, and $p99$. Every number below is reproduced by the small Python program in Appendix A, so the arithmetic is exact and self-consistent. The module quotes RAG at \$0.005/query and agents at \$0.50–\$5/trajectory with a “100× p99” — we rebuild those figures from the quadratic history cost of Module 0.0.

Where this sits. This is **Topic 1 of Module 4.0** (*From RAG Production to Agent Production*) — the number behind the opener’s headline stat “100× agent vs. RAG p99.” Its companions: Module 0.0 (the ReAct cost derivation, $N B + h \frac{N(N-1)}{2}$, which we reuse here) and Module 4.3 (cost engineering — circuit breakers and step budgets that cap the tail derived below).

Concepts covered, in order: fixed pipeline vs. variable N · deterministic cost vs. a cost *distribution* · reusing the quadratic history cost · the heavy right tail of N · percentiles $p50$ and $p99$ by nearest-rank · the ratio $p99/p50$ · why RAG has $p99/p50 \approx 1$ · provisioning and alerting on the tail · capping N (step budget / circuit breaker).

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one fixed pipeline, one variable loop

A **RAG request** is a straight pipeline: retrieve once, call the model once, return. The number of model calls is fixed at 1 no matter what the user asks. An **agent** is the ReAct loop from Module 0.0: each turn the model reads the whole transcript, decides on a tool, and the runtime appends the result — until *the model itself* emits `final_answer`. Nobody fixes N in advance; the model does, at runtime, differently for every request. That one difference is the whole document: a fixed pipeline produces a *number*; a variable loop produces a *distribution*.

The quantities. We reuse the token model of Module 0.0, with leaner constants chosen so a single RAG call lands near the module’s quoted \$0.005. They are illustrative but round, so the sums follow on paper.

Quantity	Symbol	Value here
Base context, re-sent every step (system prompt + tool schemas + the query)	B	1,000 tokens
Assistant message per step (thought + action)	a	200 tokens
Observation appended per completed step	o	1,300 tokens
History added per completed step	$h = a + o$	1,500 tokens
Input price	p_{in}	\$3 / 1M tokens
Output price	p_{out}	\$15 / 1M tokens
Wall-clock per step (1 LLM + 1 tool call)	ℓ	2 s
Steps in a RAG request	N_{RAG}	1 (fixed)
Steps in an agent run	N	variable (the model decides)

Notation. As in Module 0.0, the total input tokens for an N -step run are $T_{\text{in}}(N) = NB + h \frac{N(N-1)}{2}$ and the cost is $\text{Cost}(N) = p_{\text{in}}T_{\text{in}}(N) + p_{\text{out}}Na$. RAG is just the $N = 1$ special case. Latency is ℓN . We carry four decimals of a dollar.

Intuition

RAG is a vending machine: one coin, one can, every time — the price is on the front. An agent is a taxi with the meter running and *the passenger* deciding when to stop. Most rides are short; a few get lost and circle the block twenty times. You cannot price a taxi fleet by the median fare — you budget for the unlucky long rides, because those are the ones that empty the wallet. That “unlucky long ride” is the $p99$, and it is the whole reason this document exists.

2 Step 1 — the RAG request is a single number

A RAG request runs the loop exactly once: $N = 1$. There is no accumulated history (the triangular term $\frac{N(N-1)}{2} = 0$), so

$$c_{\text{RAG}} = \text{Cost}(1) = p_{\text{in}}B + p_{\text{out}}a = 1000 \cdot 3 \times 10^{-6} + 200 \cdot 15 \times 10^{-6} = 0.0030 + 0.0030 = \boxed{\$0.0060}.$$

Its latency is one step: $\ell \cdot 1 = 2$ s. And crucially, *every* RAG request is this same number, because the pipeline shape never changes. Run it a million times and the cost histogram is a single spike at \$0.006 and the latency histogram a single spike at 2 s. ✓ This matches the module’s “ $\approx \$0.005/\text{query}$ ” (the small gap is our round token sizes).

Mini-example — this operation in isolation

What does “ $p99/p50 \approx 1$ ” mean for RAG? Take 100 RAG requests. Sort their costs: all 100 are \$0.0060. The median ($p50$) is \$0.0060; the 99th value ($p99$) is also \$0.0060. Their ratio is exactly 1.0. A fixed pipeline has *no* tail — $p99$ tells you nothing $p50$ didn’t already. This is the comfortable world RAG-trained instincts come from, and it is exactly the world agents leave behind.

3 Step 2 — the agent run is a distribution over N

Because the model decides when to stop, N varies request to request. Easy questions resolve in two steps; hard ones take more; and a few hit loops or bad tool calls and run away. Here is a small, hand-countable population of 20 agent trajectories — their step counts, sorted:

$$N \in \{2, 2, 3, 3, 3, 4, 4, 4, 5, 5, 5, 6, 6, 7, 8, 10, 15, 20, 30, 50\}.$$

Most cluster at 2–6 steps; a thin right tail reaches 50 (a runaway loop). Apply $\text{Cost}(N)$ to each. Three worked rows, with $p_{\text{in}} = \$3 \times 10^{-6}$, $p_{\text{out}} = \$15 \times 10^{-6}$, $h = 1500$:

N	$T_{\text{in}}(N) = NB + h \frac{N(N-1)}{2}$	in \$	$T_{\text{out}} = Na$	out \$	$\text{Cost}(N)$	latency ℓN
5	$5000 + 1500 \cdot 10 = 20,000$	0.0600	1,000	0.0150	\$0.0750	10 s
20	$20000 + 1500 \cdot 190 = 305,000$	0.9150	4,000	0.0600	\$0.9750	40 s
50	$50000 + 1500 \cdot 1225 = 1,887,500$	5.6625	10,000	0.1500	\$5.8125	100 s

Sample check, $N = 5$: triangular number $\frac{5 \cdot 4}{2} = 10$, so $T_{\text{in}} = 5 \cdot 1000 + 1500 \cdot 10 = 20,000$; input = \$0.0600, output = $1000 \cdot 15 \times 10^{-6} = \0.0150 , total \$0.0750. ✓ The same model that costs \$0.0750 for an easy request costs \$5.8125 for the runaway — a 77.5× spread, from one unchanged model. The spread is not in the model; it is in N .

Intuition

Look at the $N = 50$ row: input is \$5.66 but output is only \$0.15. The runaway is not expensive because the model *wrote* a lot — it wrote almost nothing. It is expensive because it *re-read* a 1.9-million-token transcript that it built up by looping. The tail cost is the quadratic history term of Module 0.0 ($h \frac{N(N-1)}{2}$) cashed out in dollars: at $N = 50$ that term alone is $1,500 \cdot 1225 = 1.84\text{M}$ tokens.

4 Step 3 — mean, p50, and p99 by hand

Now summarise the 20 costs. The **mean** is the total divided by 20. Summing $\text{Cost}(N)$ over the population gives \$10.7730, so

$$\text{mean cost} = \frac{10.7730}{20} = \$0.5387.$$

That is $0.5387/0.0060 \approx 90\times$ the RAG cost — the module’s “\$0.50/trajectory” and “ $\sim 100\times$ ” figures, derived. The mean is already pulled up by the tail: most runs cost well under \$0.10, yet the average is \$0.54.

Percentiles tell the real story. Using the *nearest-rank* rule on the sorted costs — the p -th percentile is the value at position $k = \lceil \frac{p}{100} n \rceil$ (1-indexed) — with $n = 20$:

Percentile	rank $k = \lceil \frac{p}{100} \cdot 20 \rceil$	N at that rank	cost
p50 (median)	$\lceil 10.0 \rceil = 10$	$N = 5$	\$0.0750
p99	$\lceil 19.8 \rceil = 20$	$N = 50$	\$5.8125

The ratio is the headline number:

$$\left. \frac{p99}{p50} \right|_{\text{cost}} = \frac{5.8125}{0.0750} = 77.5\times \quad (1)$$

Do the same for **latency** ($\ell N = 2N$ seconds): the $p50$ run is $N = 5 \Rightarrow 10\text{s}$, the $p99$ run is $N = 50 \Rightarrow 100\text{s}$, so $p99/p50 = 100/10 = 10\times$. Compare RAG, where *every* request is \$0.0060 and 2s: $p99/p50 = 1.0$ for both cost and latency.

Mini-example — this operation in isolation

Nearest-rank in isolation. For $p99$ with $n = 20$: $\frac{99}{100} \cdot 20 = 19.8$, and $\lceil 19.8 \rceil = 20$ — the *largest* value in the sample. That is the point of $p99$: with 100 requests it is essentially “the worst one you will routinely see.” For RAG that worst case equals the median; for an agent it is the runaway. The percentile machinery is identical for both — the *distribution* underneath is what differs.

5 Step 4 — why “100× p99” comes from N , not a slower model

It is tempting to read “100×” as “agents use a heavier, slower, pricier model.” They do not. Every number above used the *same* per-step cost and the *same* $\ell = 2\text{s}$. The blow-up is entirely the distribution of N :

- **RAG fixes $N = 1$.** One point, no spread. $p99/p50 = 1$.
- **The agent lets the model choose N .** A heavy-tailed N (median 5, tail 50) feeds through $\text{Cost}(N)$ — which is itself *super-linear* in N — so the cost tail is stretched even further than the N tail. N spans $5 \rightarrow 50$ ($10\times$); cost spans $\$0.075 \rightarrow \5.81 ($77.5\times$). The extra factor of ~ 7.8 is the quadratic history term amplifying the tail.

So the “100× p99” is two effects stacked: a heavy tail in N , then the quadratic cost map making that tail heavier still. Swap in a cheaper model and every number scales down together — the *ratio* $p99/p50$ does not move. You cannot buy your way out of a tail; you can only cap N .

Watch out

A mean-based budget lies to you. Here the mean is \$0.54, but 14 of 20 runs cost under \$0.13 — so if you provision for the average you are over-provisioned for the body and *under*-provisioned for the tail that actually bankrupts you. Worse, the tail is where the **blast radius** lives: a runaway $N = 50$ trajectory has made 50 tool calls — 50 chances to issue a refund, send an email, or write to a database. Provision, budget, and alert on $p99$, never on the mean.

6 Step 5 — capping the tail: the step budget

The fix is not a faster model; it is a ceiling on N . A **step budget** (forward-ref: Module 4.3’s circuit breaker) caps $N \leq N_{\max}$ and aborts beyond it. Setting $N_{\max} = 10$ on our population replaces the $\{15, 20, 30, 50\}$ tail with capped runs at $N = 10$ ($\text{Cost}(10) = \$0.2625$, 20s). The $p99$ run becomes $N = 10$ instead of $N = 50$:

	p50 cost	p99 cost	p99/p50
Uncapped agent	\$0.0750	\$5.8125	77.5×
Capped at $N_{\max} = 10$	\$0.0750	\$0.2625	3.5×
RAG (fixed $N = 1$)	\$0.0060	\$0.0060	1.0×

The cap does not touch the median (most runs never hit it) but it amputates the tail: $p99$ drops 22×, from \$5.81 to \$0.26, and the worst-case latency from 100s to 20s. That is the entire economic case for step budgets and circuit breakers — they convert a heavy-tailed distribution into a bounded one, trading a little completeness on hard requests for a hard ceiling on cost, latency, and blast radius.

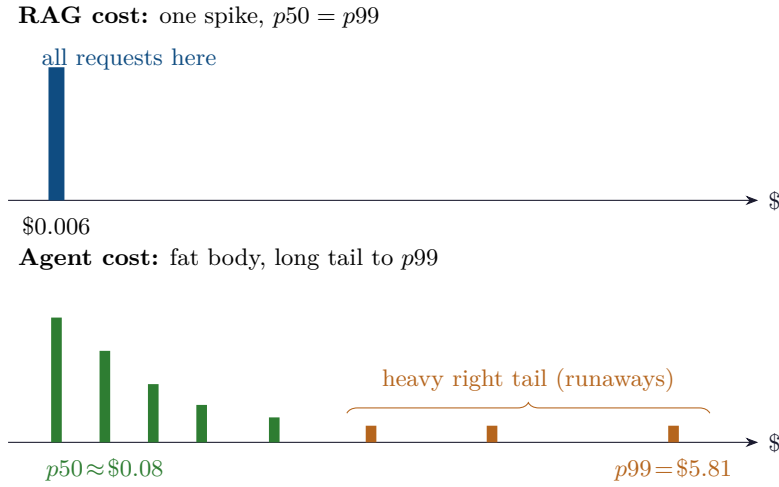
Intuition

A circuit breaker is a $p99$ instrument, not a $p50$ instrument. It almost never fires on a healthy request — it exists for the 1% that would otherwise run away. Judged by average cost it looks useless (the mean barely moves the body); judged by $p99$ it is the difference between a \$0.26 worst case and a \$5.81 one. This is why agent dashboards are built around percentiles: the lever and the metric must both live in the tail.

7 RAG vs agent in production, at a glance

Dimension	RAG production	Agent production
Model calls N	fixed at 1	variable (model decides)
Cost shape	a single number	a heavy-tailed distribution
Cost $p50$	\$0.0060	\$0.0750
Cost $p99$	\$0.0060	\$5.8125
$p99/p50$ (cost)	1.0×	77.5×
Latency $p50$ / $p99$	2s / 2s (1.0×	10s / 100s (10×
Mean vs. RAG	1×	~ 90×
Provision against	the mean (it <i>is</i> $p99$)	the $p99$ tail
Blast radius	one bad answer	up to N stateful side effects
Tail control	n/a (no tail)	step budget / circuit breaker ($N_{\max}$)

8 The distributions, drawn



RAG is one blue spike: p_{50} and p_{99} coincide. The agent is a green body of short runs with an orange tail of runaways stretching far to the right — and p_{99} lives *in the tail*, $77.5\times$ the median. Provisioning on the median (green) leaves the orange uncovered. The cap of \$6 chops the orange off.

9 Cost & p99 cheat-sheet

RAG cost (fixed, $N = 1$)	$c_{\text{RAG}} = p_{\text{in}}B + p_{\text{out}}a$
Agent cost (N steps)	$\text{Cost}(N) = p_{\text{in}}[NB + h\frac{N(N-1)}{2}] + p_{\text{out}}Na$
Latency	ℓN (RAG: $\ell \cdot 1$)
p -th percentile (nearest-rank)	value at rank $k = \lceil \frac{p}{100} n \rceil$ of the sorted sample
Tail ratio	p_{99}/p_{50} (RAG ≈ 1 ; agent 10–100 $\times$)
Why the tail blows up	heavy-tailed $N \times$ super-linear $\text{Cost}(N)$
Tail control	step budget / circuit breaker: cap $N \leq N_{\text{max}}$
Provision / budget / alert on	p_{99} , never the mean

10 An honest word about these numbers

The token sizes ($B = 1000$, $h = 1500$), the price per token, the $\ell = 2$ s per step, and the particular 20-trajectory sample are all illustrative round numbers, picked so the percentiles fall on clean ranks you can check by hand. Your real distribution of N is empirical — you get it from production traces, and it is rarely this tidy. Two things, though, are *not* illustrative. First, the **structure**: a fixed pipeline has $p_{99}/p_{50} \approx 1$ while a model-chosen N produces a heavy tail, and a super-linear cost map stretches that tail further. Second, the **lesson**: because the body and the tail diverge, the mean is a misleading budget and p_{99} is the number to provision, alert, and cap against. Change the constants and the dollar figures move; the fixed-vs-distribution gap, and the case for a step budget, do not. That gap is the thing to carry into the rest of Track 4 — it is why agents have cost SLOs, circuit breakers, and blast-radius controls at all.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the RAG cost, the agent per- N table, the mean, the $p50/p99$ percentiles and their ratio, and the effect of the step-budget cap; change B , a , o , the prices, or the `step_counts` sample to explore your own agent's economics.

```

1  import math
2  # --- token / price model (same shape as Module 0.0) ---
3  B, a, o = 1000, 200, 1300 # base context, assistant msg, observation (tokens)
4  h = a + o # history added per completed step = 1500
5  P_IN, P_OUT = 3e-6, 15e-6 # $ per input / output token ($3 / $15 per 1M)
6  L_STEP = 2.0 # seconds per agent step (1 LLM + 1 tool call)
7
8  def total_cost(N): # Cost(N) = p_in*[N*B + h*N(N-1)/2] + p_out*N*a
9      t_in = N * B + h * (N * (N - 1)) // 2
10     t_out = N * a
11     return t_in * P_IN + t_out * P_OUT
12
13 def pct(sorted_vals, p): # nearest-rank percentile, 1-indexed
14     k = max(1, math.ceil(p / 100.0 * len(sorted_vals)))
15     return sorted_vals[k - 1]
16
17 # --- RAG: fixed pipeline, N = 1 ---
18 c_rag = total_cost(1)
19 print(f"RAG cost = ${c_rag:.4f} latency = {L_STEP:.0f}s p99/p50 = 1.0x")
20
21 # --- Agent: variable N with a heavy right tail (20 trajectories) ---
22 step_counts = [2,2,3,3,3,4,4,4,5,5,5,6,6,7,8,10,15,20,30,50]
23 n = len(step_counts)
24 costs = sorted(total_cost(N) for N in step_counts)
25 lats = sorted(N * L_STEP for N in step_counts)
26
27 mean = sum(costs) / n
28 c50, c99 = pct(costs, 50), pct(costs, 99)
29 l50, l99 = pct(lats, 50), pct(lats, 99)
30 print(f"agent mean = ${mean:.4f} (= {mean/c_rag:.0f}x RAG)")
31 print(f"agent cost p50=${c50:.4f} p99=${c99:.4f} ratio={c99/c50:.1f}x")
32 print(f"agent latency p50={l50:.0f}s p99={l99:.0f}s ratio={l99/l50:.0f}x")
33
34 # --- step budget / circuit breaker: cap N <= Nmax ---
35 Nmax = 10
36 capped = sorted(total_cost(min(N, Nmax)) for N in step_counts)
37 print(f"capped(N<=10) p50=${pct(capped,50):.4f} p99=${pct(capped,99):.4f} "
38       f"ratio={pct(capped,99)/pct(capped,50):.1f}x")
39
40 # --- scaling table ---
41 print("N cost($) latency(s)")
42 for N in (1, 3, 5, 10, 20, 30, 50):
43     print(f"{N:2d} {total_cost(N):7.4f} {N*L_STEP:6.0f}")
44
45 # Expected output:
46 # RAG cost = $0.0060 latency = 2s p99/p50 = 1.0x
47 # agent mean = $0.5387 (= 90x RAG)
48 # agent cost p50=$0.0750 p99=$5.8125 ratio=77.5x
49 # agent latency p50=10s p99=100s ratio=10x
50 # capped(N<=10) p50=$0.0750 p99=$0.2625 ratio=3.5x
51 # N cost($) latency(s)
52 # 1 0.0060 2
53 # 3 0.0315 6
54 # 5 0.0750 10
55 # 10 0.2625 20
56 # 20 0.9750 40

```

57	# 30 2.1375 60
58	# 50 5.8125 100

— *end of document* —